Confidence-Aware Adaptive Verification of LLM Recommendations in Introductory Programming Tasks

Student: Lidia Davydova

Supervisor: Paolo Ciancarini

Abstract. Large language models (LLMs) are increasingly used in programming education, but their recommendations can be plausible yet incorrect, so the quality of human–AI collaboration depends on *appropriate reliance*: accepting correct recommendations and rejecting incorrect ones. Verification support can help users evaluate recommendations, and introductory programming is well suited to it because correctness can be checked through independent, execution-based evidence such as tests, traces, and counterexamples. Providing such evidence on every trial, however, introduces unnecessary interaction cost. This thesis investigates whether a simple, interpretable adaptive policy—gating verification availability on the participant's initial confidence—can reduce verification burden relative to always-available verification while preserving decision quality. A within-subject experiment (provisional $N = 60$) compares no-verification, static, and adaptive conditions on a benchmark of 18 introductory Python tasks with frozen, pre-validated Qwen3-8B recommendations balanced between correct and incorrect. Trial-level outcomes are analysed with mixed-effects logistic regression with participant and task random effects; verification burden is measured through availability, examination, and time measures and reported separately from quality outcomes. Expected contributions include trial-level evidence on the burden–quality trade-off of confidence-gated verification, a validated reproducible benchmark with an instrumented prototype, and a complete analysis pipeline.

1 Introduction and Research Problem

Large language models (LLMs) are increasingly used in programming education and software engineering to generate, explain, and debug code and to propose answers to tasks [1]–[3]. Because such systems can produce plausible but incorrect outputs, the quality of the joint human–AI outcome depends not only on model accuracy but on *appropriate reliance* [4]: accepting correct recommendations and rejecting incorrect ones. Over-reliance and under-reliance are the corresponding failures [5], [6].

Verification support can help users evaluate recommendations, and programming tasks suit it well: correctness can often be checked independently through execution, tests, traces, or counterexamples—evidence grounded in the task itself rather than in another LLM-generated justification. This matters because explanations assist only when users can interpret them and connect them to the recommendation, and simply providing more information does not automatically reduce over-reliance [6]–[9]. Moreover, one can only verify an answer one has the grounding to check [10]; introductory programming provides exactly such grounding. However, verification on every trial introduces unnecessary interaction cost, which motivates providing verification evidence *selectively* [11].

1.1 Research Problem

The research problem is how to provide verification support selectively, using only information available before the correctness of the LLM recommendation is known. The thesis investigates whether a simple adaptive policy, based primarily on the user's initial confidence, can provide independent verification support at lower burden than always-available verification, and how verification burden

trades off against decision quality. The focus is a simple, interpretable policy—not a new LLM, a new verification algorithm, or an optimisation method. The contribution is therefore not the threshold rule itself; it is the controlled empirical evaluation of whether a simple confidence gate can reduce verification exposure while preserving appropriate reliance compared with always-available and no-verification baselines.

2 Research Questions

Primary RQ. How does confidence-aware adaptive verification affect reliance on LLM recommendations in introductory programming tasks compared with always-available (static) verification and no verification?

RQ1. How do no, static, and adaptive verification affect final answer accuracy, over-reliance, and under-reliance?

RQ2. What trade-off does adaptive verification exhibit between verification burden (verification availability, evidence-examination rate, evidence-examination time) and outcome quality (final accuracy, appropriate reliance) relative to static verification?

RQ3. How do the burden and effectiveness of the adaptive policy differ across initial confidence levels and task types?

Neither task type nor task difficulty is an input to the adaptive policy (Section 5.4); both are analysed only as exploratory moderators of policy effectiveness.

A policy premise check examines whether initial confidence is associated with initial answer correctness (Section 5.5).

3 Novelty and Contributions

The novelty is *not* the observation that confidence relates to reliance on AI—miscalibration is well documented [8], [12] and taken here as a premise—but the combination of elements usually studied in isolation: (i) confidence as a per-trial *trigger* deciding *when* support is shown, rather than an outcome to be calibrated or an AI-side uncertainty signal; (ii) *independent, execution-based* evidence (tests, traces, counterexamples) rather than additional AI-generated explanations; (iii) a programming setting in which recommendations are pre-validated and objective evidence exists; and (iv) an explicit burden–quality trade-off evaluation, treating verification burden as a first-class outcome [10], [11].

The thesis is expected to contribute: (1) trial-level empirical evidence, analysed with mixed-effects models, on how a simple confidence-based policy trades verification burden against accuracy and appropriate reliance (RQ1–RQ3); (2) a validated benchmark of 18 introductory Python tasks with fixed, pre-validated LLM recommendations (balanced correct/incorrect), task-type and difficulty labels, and verification data; (3) a reproducible prototype of the three conditions with complete logging, including evidence examination; (4) an analysis pipeline for all outcome and burden measures. The contribution is deliberately not a new LLM, a new algorithm, or a tutoring system: the confidence gate is intentionally simple, and its value lies in the controlled empirical comparison against always-available and no-verification baselines in a reproducible human–AI interaction experiment.

4 State-of-the-Art

Trust and appropriate reliance. Trust in automation is classically defined as the attitude that an agent will help an individual achieve a goal under uncertainty and in risky situations, and the design goal is *appropriate* reliance rather than maximal trust [4]. In AI-assisted decision-making, users frequently fail to calibrate reliance to recommendation quality: over-reliance on incorrect advice and under-reliance on correct advice persist even when additional information about the model is available [5], [13]. Comparable miscalibration between confidence and correctness is documented for human judgment itself [12].

Explanations and cognitive forcing. Cognitive forcing functions, which compel deliberate engagement with AI advice before it is accepted, can reduce over-reliance [7]. However, meta-analytic evidence finds no reliable benefit of explanations over AI predictions alone [9], and experimental work shows that explanations reduce over-reliance only when they are interpretable and help users anticipate AI errors [6]; interventions evaluated with LLM advice similarly reduced over-reliance without improving appropriate reliance overall [8]. Because explanation-based support may itself provide additional AI-generated information, this motivates verification support grounded in independent, execution-based evidence rather than additional AI text.

Verification and its cost. Correctness alone is an insufficient evaluation criterion: human scrutiny of AI outputs is valuable but consumes time and effort, which motivates frameworks that invoke human input selectively [11]. Furthermore, one can only verify an answer one has the grounding to check [10]; intro-

ductory programming supplies such grounding—executable reference answers, tests, and traces—so objective verification evidence is available at decision time.

Confidence as a signal for adaptive support. Closest to this thesis, [8] evaluate interventions for reliance on LLM advice and find that interventions reduce over-reliance but generally fail to improve appropriate reliance, while users' confidence is poorly calibrated. The authors discuss confidence calibration as a design implication. Related work adapts support to the user's trust state [14]. Here, confidence is instead an *input signal*: a lightweight trigger deciding *when* independent support is shown, leaving the user's internal state unchanged. The documented relationship between confidence and correctness is taken as a premise: if confidence carries information about correctness, it can also inform when verification is worth its interaction cost—a burden dimension absent from calibration-oriented work and a primary outcome here (RQ2).

LLM assistance in programming and research gap. LLMs are increasingly used for code generation, debugging, and explanation in computing education [1]–[3], but can be plausible yet incorrect, so users must evaluate recommendations critically. Prior work establishes that intuition, explanation design, and decision difficulty shape reliance [6], [10], [13]; what remains unknown is whether a participant's initial confidence can serve as a lightweight trigger for selectively displaying *independent verification evidence* after an LLM recommendation in introductory programming tasks—the gap addressed here.

5 Goal of the Thesis

The goal of the thesis is to design, implement, and empirically evaluate an experimental system that provides selective verification support for LLM recommendations in introductory programming tasks. The study investigates the trade-off between decision quality and observable verification burden produced by a simple confidence-based adaptive policy relative to an always-available (static) policy.

The project schedule is provided in Section 6.

5.1 Task Domain and Benchmark

The benchmark comprises 18 introductory Python programming tasks of three types: *identify bug*, *check against specification*, and *select correct implementation*. "Write or repair code" tasks are excluded because they do not provide a sufficiently constrained response space for the primary reliance measures.

The benchmark metadata include task type, difficulty level, reference answer, fixed LLM recommendation, recommendation correctness, and task-specific verification data. The 18 LLM recommendations are balanced between nine correct and nine incorrect recommendations. Correct and incorrect recommendations are distributed across the three task types as evenly as the benchmark construction permits.

Task difficulty. Each task is assigned a difficulty level (*low*, *medium*, or *high*) before the main experiment according to a predefined rubric based on code complexity, the number of reasoning steps required, and the programming constructs involved. Difficulty ratings are checked during the pilot against pilot accuracy and response times and are frozen together with the benchmark. Difficulty is not used

as an input to the adaptive policy and is analysed only as an exploratory factor; if the pilot indicates that the ratings are unreliable, difficulty is dropped from the confirmatory analysis and retained only as descriptive metadata.

Each task has a closed-form response space that makes correctness and changes in the participant's answer well-defined. For *identify bug*, the response consists of a binary bug-presence judgment and, when a bug is present, a line number identifying its location. Optional free-text explanations are recorded only for qualitative analysis. For *check against specification*, the response is a binary `COMPLIANT`/`NOT_COMPLIANT` judgment. For *select correct implementation*, the response is an identifier corresponding to one of a fixed set of candidate implementations.

Each participant completes 18 trials, with six trials assigned to each experimental condition. Each condition contains two tasks of each of the three task types. Within each condition, three trials contain correct LLM recommendations and three trials contain incorrect LLM recommendations, with one correct and one incorrect recommendation for each task type.

Because condition membership varies within task across participants, task-level heterogeneity is not systematically confounded with the experimental condition, allowing task to be included as a random effect in the analysis models.

5.2 LLM Recommendation Specification

All LLM recommendations are generated offline before the main experiment. No LLM generation is performed during participant trials. Consequently, all participants receive the same frozen recommendation for a given task, and differences between experimental conditions cannot be attributed to runtime LLM

sampling variance.

Model and version.

The recommendation generator is a fixed local snapshot of a specific `Qwen3-8B` model artifact, an 8.2B-parameter instruction-tuned model distributed by Qwen. The experiment uses an 8-bit quantized model in GGUF format (`Q8_0`). The exact model file, model revision, quantization, inference backend, software version, and model-file SHA-256 hash are recorded and frozen before benchmark generation. The benchmark therefore refers to a specific frozen model artifact rather than to the mutable upstream `Qwen3-8B` model name.

Inference configuration.

Qwen3 is used in non-thinking mode so that generated recommendations contain only the requested answer and do not depend on a model-generated reasoning trace. The generation configuration is fixed as follows:

```
model: Qwen3-8B model artifact, 8-bit Q8_0
thinking: disabled
temperature: 0.7
top_p: 0.8
top_k: 20
max_tokens: 256
frequency_penalty: 0
```

The sampling parameters `top_k` and `frequency_penalty` are backend-dependent; their concrete semantics are those of the archived backend version.

Reproducibility.

The exact inference backend and version, command-line options, context length, and hardware configuration are archived with the benchmark. The gener-

ation seed is archived when supported by the inference backend. Because GPU inference may be nondeterministic even under a fixed seed, reproducibility of the benchmark is defined with respect to the archived model artifact and the archived generation outputs rather than bit-exact regeneration.

Hardware configuration

```
VRAM: Tesla V100 16 GB
RAM: 128 GB
```

Prompt template.

For each task, the model receives a structured prompt containing: (i) the task type and instructions, (ii) the task description and any Python code snippet, (iii) a strict output schema, and (iv) an instruction to return only the requested answer.

The prompt does not contain information about the experimental condition, verification policy, participant confidence, or intended correctness of the recommendation. The complete prompt template and all task-specific prompts are archived with the benchmark.

Candidate generation.

For each benchmark task, multiple candidate LLM recommendations are generated using a fixed inference configuration. The model receives a task-specific prompt containing the task description, required instructions, and a strict output format corresponding to the task's predefined response space. The model is instructed to solve the task and return only the requested answer, without explanations or confidence estimates.

The LLM recommendation is a task-specific structured answer expressed in the predefined response space of the task. For example, for an *identify bug*

task, it may be a predefined bug location or type (e.g., `LINE_3`); for a *check against specification* task, it may be a binary judgment such as `SATISFIES` or `DOES_NOT_SATISFY`; and for a *select correct implementation* task, it may be an implementation identifier such as `B`. The exact response representation is defined separately for each task before candidate generation.

Each model output is parsed into the task's canonical structured response representation. Outputs that cannot be parsed unambiguously are rejected and regenerated. Candidate recommendations are then evaluated against the reference answer to determine their correctness.

For each task, one recommendation is selected for the final benchmark. The benchmark contains an equal number of correct and incorrect recommendations. Correct recommendations must correspond to the reference answer. Incorrect recommendations must be objectively incorrect while remaining sufficiently plausible to a novice participant. Selection is performed before the participant study and does not depend on participant data or experimental condition.

The final recommendation is fixed for the entire study: the same task always uses the same LLM recommendation regardless of experimental condition or participant. The recommendation is stored in canonical structured form for analysis and rendered in a participant-facing form appropriate to the task type.

For correct-recommendation tasks, one valid candidate is selected. For incorrect-recommendation tasks, an incorrect candidate is selected only if it is both objectively incorrect and sufficiently plausible to a novice participant. If no suitable incorrect candidate is produced by the model, the benchmark construction protocol permits a documented minimal perturbation of a correct answer, such as an off-by-one value, a wrong operator, or another task-appropriate error. Such perturbations are recorded separately from model-generated candidates.

Validation procedure.

Recommendation correctness is first established automatically whenever an objective reference procedure is available. Depending on the task type, this may involve program execution, reference-output comparison, or predefined tests. When automatic validation is not applicable, correctness is established by comparison with the predefined reference answer.

The resulting correctness label and the plausibility of incorrect recommendations are then independently reviewed by two annotators with introductory programming expertise. Annotators do not receive information about the experimental condition or the adaptive policy. Disagreements are resolved by a third annotator according to the predefined validation procedure.

An incorrect recommendation is retained only when it is objectively incorrect and judged sufficiently plausible that its incorrectness is not immediately obvious to a novice participant. The validation procedure, candidate pool, selected candidate, correctness label, annotator decisions, and any perturbation are archived.

After validation, the benchmark is frozen. No recommendation, correctness label, prompt, model parameter, or candidate-selection decision is changed during the main experiment. Recommendation correctness is hidden from participants and is never provided to the adaptive policy. The correctness label is stored only in the analysis dataset and is not available to the participant-facing application or to the policy module during a trial.

5.3 Verification Support and Burden Instrumentation

For each task, the verification mechanism is fixed before the experiment and is identical across experimental conditions. The only manipulated factor is whether the verification evidence is made available to the participant.

Verification is task-specific and provides independent evidence that can be used to evaluate the LLM recommendation. For *identify bug* tasks, the system provides predefined tests with failing cases or concrete counterexamples that provide evidence about the presence and location of a bug. For *check against specification* tasks, the system provides specification-based test cases with expected and actual results. For *select correct implementation* tasks, the candidate implementations are executed against a predefined test suite, and the resulting test outcomes are displayed.

Verification artifacts are constructed during benchmark development by the researchers and are derived independently from the reference answer and task specification. They are not generated by the LLM used to produce the experimental recommendations. Each verification artifact has a predefined expected outcome and is validated before the main experiment.

For *identify bug* tasks, the researchers construct a set of targeted test cases or counterexamples designed to expose the specified bug and provide evidence about its location or effect. For *check against specification* tasks, the researchers construct specification-based test cases covering the relevant requirements, with expected results derived from the task specification. For *select correct implementation* tasks, the researchers construct a predefined test suite containing cases that distinguish the candidate implementations, with expected outcomes derived from the task requirements.

Each verification artifact is evaluated against the reference answer or reference implementation during benchmark validation. The verification artifacts are then independently reviewed by two annotators with introductory programming expertise to confirm that they are correct, relevant to the task, and sufficient to provide evidence for the intended judgment. Disagreements are resolved by a third annotator according to the predefined validation procedure.

After validation, the verification artifacts are frozen together with the benchmark. No test case, expected result, counterexample, or other verification artifact is modified during the main experiment.

The verification mechanism does not explicitly state whether the LLM recommendation is correct and does not provide an alternative LLM judgment. Instead, it provides task-specific evidence from which participants can assess whether the recommendation is consistent with the observed behaviour or the task requirements. The evidence therefore does not directly reveal the correctness label used by the experimenter.

Verification burden and verification cost are treated as distinct concepts. Verification cost refers to the intrinsic effort required to establish the correctness of a recommendation. Because this latent effort cannot be observed directly from participant behaviour—for example, a participant may stop before establishing correctness or may reach an incorrect conclusion—the study does not claim to measure intrinsic verification cost.

Instead, the experiment measures observable verification burden imposed by the system and experienced by the participant. The burden-related measures are strictly separated by level:

- *verification availability*: whether verification evidence is made available to

the participant (system-level);

- *evidence examination*: whether the participant actually engages with the available evidence (participant-level);
- *evidence examination time*: the time the participant spends interacting with the evidence panel;
- *post-availability decision time*: the time from the moment evidence is made available to the final judgment, regardless of whether the evidence is examined (defined only on verification-available trials);
- *total decision time*: the time from presentation of the LLM recommendation to the final judgment;
- *post-evidence answer change*: whether the participant changes the initial answer after the evidence stage.

These terms are used consistently throughout the proposal: *availability* refers to the system, *examination* refers to the participant, and the two time measures are distinct constructs. The term *verification time* is deliberately avoided because it conflates system-level provision of evidence with participant-level use of evidence.

The experimental application records evidence-panel visibility events, activation and dismissal timestamps, scrolling and interaction events, and the timestamp of the final judgment. Evidence examination is operationalised using a rule specified before the main experiment: a trial is classified as *evidence-examined* if the evidence panel remains active for at least δ seconds or if the participant performs a predefined interaction with the evidence panel. The value of δ is fixed

during the pilot and frozen before the main experiment.

This instrumentation distinguishes system-level verification availability from participant-level evidence use. Evidence that is made available but not examined is still counted as verification availability, but is classified as non-examined at the participant level. Consequently, the adaptive condition can be evaluated not only by whether verification was made available less often than under static verification, but also by whether participants actually used the evidence when it was provided.

5.4 Adaptive Verification Policy

The adaptive policy receives only the participant's initial confidence $c \in [0, 100]$. Verification is made available when

$$c < \tau,$$

where τ is fixed before the main experiment. The threshold is either specified a priori or selected using a separate pilot dataset. Pilot participants are disjoint from main-experiment participants, and pilot data are excluded from all main-experiment analyses; τ is not estimated, optimised, or modified using data from the main experiment.

The policy does not receive the correctness of the LLM recommendation. Task type and task difficulty are not policy inputs; they are retained as experimental metadata and analysed only as exploratory factors (RQ3). The policy therefore uses a single input—initial confidence—and nothing else.

The primary purpose of the policy is to test whether a simple confidence-based gating rule can reduce verification availability relative to an always-available

(static) policy while maintaining decision quality. The policy is deliberately minimal; its scientific value lies in the controlled comparison against the always-available and no-verification baselines (RQ1–RQ2), not in the rule itself.

5.5 Policy Premise Check

The adaptive verification policy is based on the premise that initial confidence provides information about the probability that the participant's initial answer is correct. This premise is examined independently of the policy decision. Specifically, initial-answer correctness is modelled as a function of initial confidence using the mixed-effects structure specified in Section 5.9, with initial confidence scaled to the $[0, 1]$ range and participant and task included as random effects.

The purpose of this analysis is to determine whether the assumed relationship between confidence and initial-answer correctness is present in the collected data. A positive association would provide empirical support for the assumption that initial confidence contains information useful for confidence-based gating: participants with lower initial confidence would, on average, be more likely to hold an incorrect initial answer and could therefore be more likely to benefit from verification. A weak or absent association would indicate a boundary condition of the confidence-gating approach, suggesting that initial confidence provides limited information for selectively allocating verification.

This analysis does not determine or modify the verification threshold τ. The threshold is fixed before the main experiment according to the procedure specified in the experimental design and remains unchanged throughout data collection. The premise check is therefore an interpretive analysis rather than a policy-tuning

procedure.

The realised verification availability of the adaptive condition is reported as an outcome of the fixed policy. In particular, the analysis reports the proportion of trials for which the confidence threshold triggered verification and examines how this availability relates to initial confidence. This allows the study to assess whether the fixed policy actually reduces verification availability relative to static verification without using the observed main experiment data to optimise the policy.

5.6 Participants, Recruitment, and Sample Size

Population.

The target population is university students aged 18 or over who are enrolled in or have completed at least one introductory programming course in any programming language. The benchmark requires only basic Python reading, control-flow, and input/output knowledge.

Inclusion criteria.

Participants must:

1. be at least 18 years old;
2. have completed or be currently enrolled in an introductory programming course, based on self-report;
3. demonstrate basic Python-reading competence using a short screening quiz;

Exclusion criteria.

Participants are excluded if they:

1. fail the screening quiz according to the pre-registered criterion;

2. report prior exposure to the exact benchmark tasks; or

3. experience technical problems that prevent completion of the experiment.

Recruitment.

Participants are recruited through departmental mailing lists, course announcements, and student groups. Compensation may consist of course credit or a small voucher according to local institutional practice. Participation is voluntary and requires informed consent. Participants may withdraw at any time. Experimental logs are pseudonymised or anonymised as required by the applicable institutional and data-protection procedures.

Sample size.

The provisional target sample is $N = 60$ participants, with a minimum feasible sample of $N = 36$. The final target sample size is determined by the simulation-based power analysis described below and is fixed before preregistration; the provisional values serve feasibility planning. With a provisional sample of $N = 60$, each participant contributes 18 trials, yielding up to 1,080 trial-level records. These records are clustered within participants and tasks and are therefore not treated as independent observations.

The benchmark contains nine correct and nine incorrect LLM recommendations. Task assignment is counterbalanced across participants as specified in Section 5.1 so that, within each condition, approximately half of the trials contain correct recommendations and half contain incorrect recommendations. Under the planned balanced assignment, each condition therefore contributes approximately 180 observations with correct recommendations and 180 observations with incorrect recommendations at the provisional target sample size.

The six trials per condition are not treated as independent replications at the participant level. Instead, the primary analysis uses trial-level observations in mixed-effects models with participant and task included as random effects. The sample size is therefore justified with respect to the planned mixed-effects analysis rather than by treating the repeated trials within a participant as independent observations.

Because each participant contributes only three trials with incorrect recommendations and three trials with correct recommendations per condition, participant-level estimates of over- and under-reliance are expected to be unstable. These measures are therefore treated as trial-level binary outcomes in the primary analysis, while aggregated rates are reported only as descriptive summaries.

A simulation-based power analysis is conducted before preregistration using the planned mixed-effects logistic models and variance components estimated from the pilot data. The simulation varies plausible effect sizes, participant variance, task variance, and the expected proportions of correct and incorrect LLM recommendations. The final target sample size, together with the confidence threshold τ, the examination threshold δ, and the decision-time budget B, is fixed after the pilot and before main data collection. Pilot participants are disjoint from main-experiment participants, and pilot data are excluded from all main-experiment analyses. Main-experiment data are not used to increase the sample size unless an explicitly preregistered stopping or recruitment rule requires it.

A complete session is expected to take approximately 45–60 minutes, including instructions, experimental trials, and the demographic questionnaire.

5.7 Experimental Design and Procedure

Three conditions are compared:

1. *No verification*: the participant receives the LLM recommendation but no verification evidence is provided.

2. *Static verification (always available)*: verification evidence is always made available after the LLM recommendation.

3. *Adaptive verification*: verification evidence is made available only when the participant's initial confidence satisfies $c < \tau$.

The verification mechanism is identical in all three conditions. Only the display policy is manipulated.

Each participant completes all 18 benchmark tasks. The benchmark contains six tasks of each type: *identify bug*, *check against specification*, and *select correct implementation*. Within each task type, three tasks contain correct LLM recommendations and three contain incorrect recommendations.

Two tasks of each type are assigned to each experimental condition. Within each condition, one task of each type has a correct LLM recommendation and one has an incorrect LLM recommendation. Thus, each participant completes six trials per condition, with three correct and three incorrect LLM recommendations overall. Condition-to-task assignment is counterbalanced across participants as specified in Section 5.1.

Each trial follows the same sequence:

Task

↓

Initial answer

↓

Confidence

↓

LLM recommendation

↓

Condition-specific verification

↓

Final judgment

The participant first provides an answer independently. The participant then reports confidence on a 0–100 scale. The fixed benchmark recommendation is then displayed. Depending on the experimental condition, verification evidence is either unavailable, always available, or made available according to the confidence policy.

The participant subsequently records a final answer. The final answer may confirm the initial answer or change it. The application does not reveal the predetermined correctness label of the recommendation. The correctness label is stored only in the analysis dataset and is not available to the participant-facing application or to the policy module during a trial.

For each trial, the system records the task identifier, task type, difficulty, initial answer, initial confidence, fixed recommendation, condition, adaptive-policy decision, recommendation correctness label for analysis only, evidence availability, evidence-panel events, evidence examination status, examination duration,

post-availability decision time, final answer, answer-change status, and relevant timestamps.

5.8 Evaluation Metrics

The evaluation metrics are organised into primary outcome measures, reliance measures, and verification-related measures. The primary outcome measures are final-answer correctness and reliance on the LLM recommendation. Reliance is analysed at the trial level using the participant's acceptance of the LLM recommendation, while over-reliance and under-reliance are derived from this measure conditional on the objective correctness of the recommendation.

Final-answer accuracy is the proportion of trials on which the participant's final structured answer is correct according to the benchmark reference answer.

For each trial, LLM recommendation acceptance is defined as

$$\text{AcceptLLM}_i = I(\text{Final}_i = \text{LLM}_i),$$

where equality is evaluated on the canonical structured response representation defined in Section 5.1. Thus, acceptance is determined exactly rather than by manual semantic comparison.

The primary statistical analysis models AcceptLLM across all trial-level observations using a mixed-effects logistic regression with experimental condition, recommendation correctness, and their interaction as fixed effects and participant and task as random intercepts. The condition $\times$ recommendation-correctness interaction is used to assess whether the effect of verification differs between correct and incorrect LLM recommendations. Condition contrasts among incorrect recommendations quantify the effect on over-reliance, whereas condition contrasts

among correct recommendations quantify the effect on under-reliance.

For descriptive reporting, over-reliance and under-reliance are defined as follows:

$$\text{OverReliance} = \frac{N(\text{LLM incorrect} \wedge \text{Final} = \text{LLM})}{N(\text{LLM incorrect})}$$

$$\text{UnderReliance} = \frac{N(\text{LLM correct} \wedge \text{Final} \neq \text{LLM})}{N(\text{LLM correct})}$$

Over-reliance therefore represents the proportion of incorrect LLM recommendations that are accepted, whereas under-reliance represents the proportion of correct LLM recommendations that are rejected. These rates are reported descriptively by experimental condition; inferential comparisons are obtained from the primary mixed-effects model rather than from separate subset models.

Appropriate reliance is the proportion of trials on which the participant accepts a correct LLM recommendation or rejects an incorrect one:

$$\text{AppropriateReliance} = \frac{\begin{array}{c} N(\text{LLM correct} \wedge \text{Final} = \text{LLM}) \\ + N(\text{LLM incorrect} \wedge \text{Final} \neq \text{LLM}) \end{array}}{N_{\text{trials}}}.$$

Decision-change rate is the proportion of trials on which the final answer differs from the initial answer. Changes are classified along two independent dimensions: (i) a correctness transition relative to the benchmark reference answer (correct→correct, correct→incorrect, incorrect→correct, incorrect→incorrect), which is used for outcome analysis; and (ii) the relation to the LLM recommendation (toward or away from the recommendation), which is reported descriptively. The two classifications are distinct. For example, a change from a correct initial

answer to a different incorrect answer may move away from an incorrect recommendation while still producing an incorrect final outcome.

5.8.1 Verification Burden

Verification burden is operationalised using directly observable measures. The primary system-level measure is the *verification availability rate* (VAR):

$$\text{VAR} = \frac{N(\text{verification available})}{N_{\text{trials}}}.$$

For the No Verification condition, VAR is zero. For the Static Verification condition, VAR is one apart from pre-specified exclusions. For the Adaptive Verification condition, VAR depends on the participant's confidence and the fixed threshold τ.

Participant-level burden is characterised by:

- evidence-examination rate;
- evidence-examination time;
- post-availability decision time;
- total decision time;
- proportion of displayed evidence that was not examined;
- proportion of trials followed by a final-answer change.

These measures are reported separately rather than combined into an aggregate burden score.

5.8.2 Verification Success

Verification success is evaluated relative to a pre-specified decision-time budget B. The budget is an analysis-time censoring threshold on the post-availability decision time rather than an interface timeout: participants are not forced to stop at B. The value of B is calibrated using the pilot, for example near the 90th percentile of observed post-availability decision times, and is frozen before the main experiment.

Let V_i denote the participant's implicit verdict about the LLM recommendation on trial i:

$$V_i = \begin{cases} \text{accept}, & \text{if Final}_i = \text{LLM}_i, \\ \text{reject}, & \text{otherwise.} \end{cases}$$

Let C_i indicate whether the LLM recommendation is objectively correct. A verdict is considered correct when the participant accepts a correct recommendation or rejects an incorrect recommendation.

Let E_i indicate that verification evidence was available and examined on trial i, as defined in Section 5.3, and let D_i^{post} denote the post-availability decision time. The verification success indicator is

$$S_{B,i} = I\left(V_i \text{ is correct} \wedge D_i^{\text{post}} \leq B\right),$$

evaluated only on trials with $E_i = 1$. The verification success rate is

$$\text{VSR} = \frac{N(S_B = 1)}{N(E = 1)}.$$

VSR is therefore a conditional measure characterising whether participants

who engaged with the verification evidence reached a correct verdict within the decision-time budget. Trials in which evidence was available but not examined are excluded from the primary VSR because the evidence was not actually engaged with. The same statistic computed over all verification-available trials is reported as a descriptive supplement.

VSR does not establish that verification caused the correct decision, because a participant may already have held the correct judgment before examining the evidence.

For incorrect recommendations, the verification failure rate is

$$\text{VFR}_{\text{incorrect}} = \frac{N(\text{LLM incorrect} \wedge E = 1 \wedge S_B = 0)}{N(\text{LLM incorrect} \wedge E = 1)}.$$

This measures the proportion of incorrect recommendations that were not rejected within the decision-time budget despite the evidence being available and examined.

Verification success and verification burden are reported separately. No aggregate quality–burden score is constructed.

5.9 Statistical Analysis

The primary analyses are conducted at the trial level using mixed-effects regression models. Repeated observations from the same participant and the same benchmark task are not treated as independent. Participant and task are therefore included as random effects to account for participant-level and task-level heterogeneity. Because task–condition assignments are counterbalanced across participants (Section 5.1), task characteristics are not systematically associated with a particular experimental condition.

For binary outcomes, mixed-effects logistic regression models are used. The general model specification is

$$\text{logit}(P(Y_{ij} = 1)) = \beta_0 + \boldsymbol{\beta}^{\mathsf{T}}\mathbf{X}_{ij} + u_i + v_j,$$

where Y_{ij} is a binary outcome for participant i on task j, $\mathbf{X}_{ij}$ represents the fixed-effect predictors, u_i is a participant-level random intercept, and v_j is a task-level random intercept.

The primary predictor is experimental condition, with No Verification as the reference condition. Planned contrasts compare Adaptive Verification with No Verification and Static Verification with No Verification. A further comparison between Adaptive and Static Verification evaluates whether the adaptive policy produces comparable outcomes while making verification available on fewer trials.

Recommendation correctness is included as a planned factor for reliance analyses because the effect of verification is expected to differ between correct and incorrect LLM recommendations. The interaction between experimental condition and recommendation correctness is therefore a primary term in the reliance analysis. Task type and task difficulty may be included as exploratory moderators. Neither task type nor task difficulty is used as an input to the adaptive verification policy, and analyses involving these variables are reported as exploratory.

The primary outcome is final-answer correctness, modelled at the trial level using a mixed-effects logistic regression with experimental condition as the primary predictor and participant and task as random intercepts. Planned condition contrasts assess differences in final-answer correctness between the experimental conditions.

Reliance on the LLM recommendation is analysed using a single mixed-effects logistic regression on all trial-level observations. For each trial, acceptance of the LLM recommendation is defined as

$$\text{AcceptLLM}_{ij} = I(\text{Final}_{ij} = \text{LLM}_j),$$

where equality is evaluated using the canonical structured response representation defined in Section 5.1. The primary reliance model is

$$\text{logit}\big(P(\text{AcceptLLM}_{ij} = 1)\big) = \beta_0 + \beta_1 \text{Condition}_{ij} + \beta_2 \text{RecCorrect}_j + \beta_3 \left(\text{Condition}_{ij} \times \text{RecCorrect}_j\right) + u_i + v_j.$$

The condition-by-recommendation-correctness interaction tests whether the effect of verification on acceptance differs between correct and incorrect LLM recommendations. For incorrect recommendations, acceptance corresponds to over-reliance; therefore, the planned condition contrasts within this level quantify differences in over-reliance. For correct recommendations, rejection corresponds to under-reliance; therefore, the planned condition contrasts within this level quantify differences in under-reliance. This single model is preferred to separate regression models fitted to correct and incorrect recommendation subsets because it uses all trial-level observations and estimates the condition-by-correctness interaction directly.

Aggregate over-reliance, under-reliance, and appropriate-reliance rates are reported descriptively by experimental condition. Over-reliance is the proportion of incorrect recommendations that are accepted, whereas under-reliance is the proportion of correct recommendations that are rejected. Appropriate reliance is the proportion of trials on which the participant accepts a correct recommendation or rejects an incorrect recommendation.

The trial-level indicator of appropriate reliance may additionally be analysed as a secondary mixed-effects logistic outcome. Answer-change rate is analysed separately as a binary outcome indicating whether the final answer differs from the initial answer. Where appropriate, answer changes are also classified by their correctness transition relative to the benchmark reference answer.

The primary analysis of verification availability compares the proportion of trials in which verification is made available under the Static and Adaptive conditions. Because verification availability is determined directly by the fixed adaptive policy, this analysis describes the realised behaviour of the policy rather than estimating or optimising its threshold. Verification availability in the No Verification condition is structurally zero and is therefore not included in the Static-versus-Adaptive comparison.

The relationship between initial confidence and initial-answer correctness is examined separately as the policy premise check described in Section 5.5. Initial-answer correctness is modelled using a mixed-effects logistic regression:

$$\operatorname{logit}\big(P(Y_{ij}^{\text{init}} = 1)\big) = \beta_0 + \beta_1 c_{ij} + u_i + v_j,$$

where $Y^{\text{init}} * ij$ indicates whether the initial answer of participant i on task j is correct and $c * ij$ is the reported initial confidence scaled to the $[0, 1]$ range. The coefficients u_i and v_j represent participant- and task-level random intercepts, respectively. Confidence is scaled before analysis; no further transformation is applied. This analysis evaluates whether initial confidence contains information about initial-answer correctness without using the result to modify the adaptive threshold τ.

The realised verification availability of the adaptive condition is also re-

ported as a descriptive outcome. In particular, the proportion of adaptive trials for which the fixed confidence threshold triggers verification is reported together with its distribution across initial-confidence levels. These quantities are not used to optimise τ using main-experiment data.

Time-based outcomes, including post-availability decision time and total decision time, are analysed separately because their distributions may be non-normal and right-skewed. Where appropriate, log-transformed times or generalized mixed-effects models with an appropriate response distribution are used. Trials with technically invalid timing measurements are excluded according to a predefined data-cleaning rule. Analyses of verification examination time and post-availability decision time are restricted to trials on which verification evidence was made available.

The verification success rate is treated as a secondary, conditional outcome because it is defined only for trials on which verification evidence was available and examined. Its interpretation and calculation are specified in Section 5.8.2. The analysis does not treat verification success as an independent causal effect of verification.

All primary hypotheses, contrasts, model specifications, exclusion rules, and coding of outcome variables are defined before analysis of the main experimental data. Pilot participants and pilot observations are excluded from all main-experiment analyses. The significance level and procedure for reporting confidence intervals and effect sizes are specified in the preregistration. Exploratory analyses are reported separately from the primary confirmatory analyses.

5.10 Expected Software Artefacts

The thesis will deliver:

- a frozen benchmark containing 18 introductory Python tasks;
- structured task metadata including type, difficulty, reference answer, fixed LLM recommendation, and recommendation correctness;
- the archived Qwen3-8B 8-bit model specification, exact model artifact identifier/hash, inference configuration, prompts, candidate generations, and validation records;
- the experimental web application;
- the execution-based verification module;
- the configurable confidence-based policy module;
- structured experimental logging including evidence availability, evidence examination, examination time, post-availability decision time, and total decision time;
- the frozen benchmark-generation and validation scripts;
- the statistical analysis scripts implementing the pre-registered mixed-effects models; and
- documentation sufficient to reproduce benchmark generation, experimental execution, and statistical analysis.

6 Detailed Work Plan and Schedule

The work is planned from October 2026 to mid-May 2027, with the main objective of submitting a research paper by 15 May 2027; the thesis is then extended with additional documentation, analysis, and discussion.

TABLE 6.1
Work plan and schedule

Phase and activities	**Period**	**Deliverable / milestone**
1. Literature review and research design: systematic study of appropriate reliance, verification cost, human–AI interaction, confidence-aware support, and LLM assistance in programming; completion of the bibliography	October 2026	Annotated bibliography; refined research gap and research questions
2. Finalisation of the experimental design: operational definitions, hypotheses, metrics, burden–cost distinction, confidence-based policy, statistical analysis plan, and power-simulation setup	1–15 November 2026	Draft protocol and analysis plan

Phase and activities	**Period**	**Deliverable / milestone**
3. Benchmark construction and LLM recommendations: selection of 18 introductory Python tasks, categorisation, difficulty assessment, reference-answer validation, offline generation and validation of recommendations (9 correct / 9 incorrect), and task-specific verification evidence	15–30 November 2026	Validated benchmark with frozen recommendations (M2)
4. Prototype implementation: participant interface, task manager, fixed recommendation module, verification module, adaptive policy, counterbalancing procedure, and experiment logger including evidence-examination logging	December 2026	Functional experimental prototype (M3)
5. Ethics, pilot, and protocol freeze: ethics approval; pilot recruitment and pilot testing of task clarity, interface usability, logging quality, confidence elicitation, and calibration of the confidence threshold τ, the examination threshold δ, and the decision-time budget B; simulation-based power analysis; preregistration	January 2027	Frozen configuration (τ, δ, B, N) and registered protocol (M4)

Phase and activities	Period	Deliverable / milestone
6. Main experiment: recruitment until the preregistered target sample is reached (minimum analysable $N = 36$), counterbalanced assignment, data collection, monitoring of exclusions, quality control	Late January–February 2027	Complete and anonymised trial dataset (M5)
7. Data analysis: data cleaning, descriptive statistics, mixed-effects models for accuracy and reliance, burden and examination analysis, premise check, trade-off comparison of static and adaptive verification	February–March 2027	Reproducible analysis scripts and preliminary results (M6)
8. Paper preparation: writing the methods, results, discussion, limitations, and conclusion; tables and figures; internal revision and proofreading	March–April 2027	Complete paper draft and revised manuscript (M7)
9. Paper submission: final revision, formatting, reference checking, supplementary materials, submission	By 15 May 2027	Submitted research paper (M8)
10. Thesis continuation and integration: expansion of the paper into the thesis, documentation of the software artefacts, extended analysis, final thesis version	After 15 May 2027	Extended thesis manuscript and project documentation

7 References

[1] Q. Ma, H. Shen, K. Koedinger, and S. T. Wu, "How to teach programming in the ai era? using llms as a teachable agent for debugging," in *Artificial Intelligence in Education: 25th International Conference, AIED 2024, Recife, Brazil, July 8–12, 2024, Proceedings, Part I*, Recife, Brazil: Springer-Verlag, 2024, pp. 265–279, ISBN: 978-3-031-64301-9. DOI: 10.1007/978-3-031-64302-6_19.

[2] D. Cambaz and X. Zhang, "Use of ai-driven code generation models in teaching and learning programming: A systematic literature review," in *Proceedings of the 55th ACM Technical Symposium on Computer Science Education V. 1*, ser. SIGCSE 2024, Portland, OR, USA: Association for Computing Machinery, 2024, pp. 172–178, ISBN: 9798400704239. DOI: 10.1145/3626252.3630958.

[3] P. Denny, J. Prather, B. A. Becker, *et al.*, "Computing education in the era of generative ai," *Commun. ACM*, vol. 67, no. 2, pp. 56–67, Jan. 2024, ISSN: 0001-0782. DOI: 10.1145/3624720.

[4] J. D. Lee and K. A. See, "Trust in automation: Designing for appropriate reliance," *Human Factors*, vol. 46, no. 1, pp. 50–80, 2004. DOI: 10.1518/hfes.46.1.50_30392.

[5] F. Poursabzi-Sangdeh, D. G. Goldstein, J. M. Hofman, J. Wortman Vaughan, and H. Wallach, "Manipulating and measuring model interpretability," in *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*, ser. CHI '21, Yokohama, Japan: Association for Computing

Machinery, 2021, ISBN: 9781450380966. DOI: 10.1145/3411764.3445315.

[6] H. Vasconcelos, M. Jörke, M. Grunde-McLaughlin, T. Gerstenberg, M. Bernstein, and R. Krishna, "Explanations can reduce overreliance on ai systems during decision-making," *Proceedings of the ACM on Human-Computer Interaction*, vol. 7, pp. 1–38, 2022. DOI: 10.1145/3579605.

[7] Z. Buçinca, M. B. Malaya, and K. Z. Gajos, "To trust or to think: Cognitive forcing functions can reduce overreliance on ai in ai-assisted decision-making," *Proc. ACM Hum.-Comput. Interact.*, vol. 5, no. CSCW1, Apr. 2021. DOI: 10.1145/3449287.

[8] J. Y. Bo, S. Wan, and A. Anderson, "To rely or not to rely? evaluating interventions for appropriate reliance on large language models," in *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, ser. CHI '25, New York, NY, USA: Association for Computing Machinery, 2025, ISBN: 9798400713941. DOI: 10.1145/3706598.3714097.

[9] M. Schemmer, P. Hemmer, M. Nitsche, N. Kühl, and M. Vössing, "A meta-analysis of the utility of explainable artificial intelligence in human-ai decision-making," in *Proceedings of the 2022 AAAI/ACM Conference on AI, Ethics, and Society*, ser. AIES '22, Oxford, United Kingdom: Association for Computing Machinery, 2022, pp. 617–626, ISBN: 9781450392471. DOI: 10.1145/3514094.3534128.

[10] Z. T. Zhang, F. Buchner, Y. Liu, and A. Butz, "You can only verify when you know the answer: Feature-based explanations reduce overreliance on ai for easy decisions, but not for hard ones," in *Proceedings of Mensch Und Computer 2024*, ser. MuC '24, Karlsruhe, Germany: Association for

Computing Machinery, 2024, pp. 156–170, ISBN: 9798400709982. DOI: 10.1145/3670653.3670660.

[11] C. Fanconi and M. van der Schaar, *Cascaded language models for cost-effective human-ai decision-making*, 2025. arXiv: 2506.11887 [cs.AI]. [Online]. Available: https://arxiv.org/abs/2506.11887.

[12] J. Kruger and D. Dunning, "Unskilled and unaware of it: How difficulties in recognizing one's own incompetence lead to inflated self-assessments," *Journal of Personality and Social Psychology*, vol. 77, no. 6, pp. 1121–1134, 1999. DOI: 10.1037/0022-3514.77.6.1121.

[13] V. Chen, Q. V. Liao, J. Wortman Vaughan, and G. Bansal, "Understanding the role of human intuition on reliance in human-ai decision-making with explanations," *Proc. ACM Hum.-Comput. Interact.*, vol. 7, no. CSCW2, Oct. 2023. DOI: 10.1145/3610219.

[14] T. Srinivasan and J. Thomason, "Adjust for trust: Mitigating trust-induced inappropriate reliance on ai assistance," in *Proceedings of the 31st International Conference on Intelligent User Interfaces*, ser. IUI '26, New York, NY, USA: Association for Computing Machinery, 2026, pp. 1883–1900, ISBN: 9798400719844. DOI: 10.1145/3742413.3789136.